RAZVOJ MOBILNIH APLIKACIJA I SERVISA

Predavanje 3

Kotlin II · Hello Android · Uvod u Jetpack Compose

Akademska 2026/27 · Fakultet elektrotehnike, Univerzitet u Tuzli

Današnji primjer: model podataka aplikacije za evidenciju prisustva i njen prvi ekran

DANAŠNJI PLAN

Prvo model, zatim ekran

DIO 1 ·

Kotlin II

- Klase i data klase
- Enum i sealed tipovi
- Interfejsi, ekstenzije
- Parsiranje teksta
- Operacije nad kolekcijama

DIO 2 ·

Hello Android

- Šta je Android aplikacija
- SDK verzije kao odluka
- Obilazak projekta
- Prvo pokretanje
- Android u 2026.

DIO 3 ·

Uvod u Compose

- Deklarativni UI
- Composable funkcije
- Column, Row, Box

KVIZ · PREDAVANJE 2

Šta ispisuju ove tri linije?

```
val nickname: String? = null
val grades = listOf(7, 9, 6, 10)

println(nickname?.length ?: 0)
println(grades.filter { it >= 9 })
println(grades.map { it * 10 }.last())
```

Zapišite tri odgovora. Jedna minuta.

Zatim: šta znači **it** unutar vitičastih zagrada?

ODGOVOR

0 · [9, 10] · 100

```
val nickname: String? = null
val grades = listOf(7, 9, 6, 10)


println(nickname?.length ?: 0)
// 0 – safe call daje null, Elvis daje 0

println(grades.filter { it >= 9 })
// [9, 10] – it = jedan element

println(grades.map { it * 10 }.last())
// 100
```

it je trenutni element liste.

Primijetite: nema zagrada oko lambde.

PONAVLJANJE · FILTER I MAP

filter zadržava elemente, map ih transformiše

`grades.filter { it >= 9 }`



Lambda vraća **Boolean**: zadrži ili odbaci. Isti tip, lista iste veličine ili **manja**.

`grades.map { it * 10 }`



Lambda vraća **novu vrijednost**. Lista je **uvijek iste veličine**, tip se može promijeniti.

```
// lanac: prvo filter, zatim map
checkIns.filter { it.minute > 615 }.map { it.student.name }
// List<CheckIn> → List<CheckIn> → List<String>: imena zakašnjelih
```

it = trenutni element. Isto kao **{ g -> g >= 9 }**.

TRAILING LAMBDA

Ako je zadnji argument lambda, ide izvan ()

```
val grades = listOf(7, 9, 6, 10)
// 1. lambda kao običan argument
grades.filter({ it >= 9 })

// 2. zadnji argument: izmjesti je van
grades.filter() { it >= 9 }

// 3. jedini argument: izostavi prazne ()
grades.filter { it >= 9 }

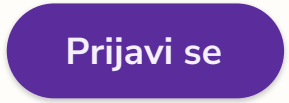
// s još jednim argumentom ispred
grades.fold(0) { acc, g -> acc + g }
```

Sve tri linije rade **potpuno isto**. Svi pišu liniju 3.

Compose je napravljen za ovaj oblik:

```
Button(onClick = { checkIn() }) {
    Text("Prijavi se")
}
Column { Text("A"); Text("B") }
```

NA EKRANU



Lambda u onClick se izvršava tek na dodir.

DIO 1

Kotlin II: modeliramo našu aplikaciju

Jedan primjer kroz cijelo predavanje. Svaki koncept dodaje jedan dio.

`Student` `CheckIn` `Window` `CheckInStatus` `CheckInRule`

1 · KLASE

Klasa objedinjuje podatke i ponašanje

- **Primarni konstruktor** u zaglavlju
- **val / var** u njemu = svojstva
- **Zadane vrijednosti** = opcioni argumenti
- **init** se izvršava pri kreiranju objekta
- **private** = vidljivo samo unutar klase
- Bez ključne riječi **new**

Android: ViewModeli i repozitoriji su klase.

```
class Lecture(
    val code: String,
    val room: String = "A1"
) {
    private var checkIns = 0

    init { println("Predavanje $code kreirano") }

    fun checkIn() { checkIns++ }
    fun count() = checkIns
}

val l3 = Lecture("RMAS-3")  // room = "A1"
l3.checkIn()
// l3.checkIns = 50  // Greška: private
```

KVIZ · KLASE

Šta se ispisuje i kojim redom?

```
class Student(val name: String) {
    init { println("1: $name") }
    val upper = name.uppercase()
    init { println("2: $upper") }
}

fun main() {
    println("start")
    Student("Lejla")
}
```

Bonus: šta ako drugi **init** dođe *prije* linije sa **upper**?

ODGOVOR

Odozgo prema dolje, redom kojim je napisano

```
start
1: Lejla
2: LEJLA
```

init blokovi i inicijalizatori svojstava izvršavaju se **redom kojim su napisani**.

Bonus: **greška pri kompajliranju**. upper se koristi prije inicijalizacije.

2 · DATA KLASE

Klasa koja samo čuva podatke

Jedna riječ, **data**, i kompajler za vas piše:

- **toString()** čitljiv ispis
- **equals()** poredi vrijednosti
- **copy()** novi objekat, neka polja izmijenjena
- **destrukturiranje** val (id, name) = s

Android: UI stanje je data klasa. Ažurira se sa **state.copy(isLoading = false)**.

```
data class Student(val id: String, val name: String)

data class CheckIn(
    val student: Student,
    val minute: Int    // 10:07 = 607
)

val amar = Student("IB101", "Amar Hodžić")
val first = CheckIn(amar, 607)
val fixed = first.copy(minute = 612)

println(fixed)
// CheckIn(student=Student(id=IB101, ...), minute=612)
val (id, name) = amar
```

KVIZ · DATA KLASE

true ili false? I šta se ispisuje?

```
class A(val name: String)
data class B(val name: String)

println(A("Amar") == A("Amar"))  // 1
println(B("Amar") == B("Amar"))  // 2
println(A("Amar"))               // 3
println(B("Amar"))               // 4
```

Četiri linije, četiri odgovora.

ODGOVOR

false · true · A@1b6d3586 · B(name=Amar)

Obična klasa

== pita: da li je to **isti objekat** u memoriji? Dva odvojena objekta, pa false. Ispis daje ime klase i hash.

Data klasa

== poredi **vrijednosti** svojstava iz konstruktora. Isto ime, pa true. Ispis prikazuje vrijednosti.

Zašto je bitno: Compose poredi staro i novo stanje da odluči šta ponovo iscrtati.

3 · ENUM KLASE

Fiksan skup vrijednosti

- Samo ove vrijednosti, nema tipfelera
- Svaka vrijednost je **jedan objekat**
- Vrijednosti mogu imati **svojstva**
- **entries** = sve vrijednosti kao lista
- **name**, **ordinal** su ugrađeni
- Savršeno radi sa **when**

Android: tabovi, filteri, sortiranja. Naša tri termina za prijavu.

```
// Profesor otvara svaki termin jednim dodirom.
// Nema fiksnih vremena: enum ih samo imenuje.
enum class Window(val label: String) {
    START("Početak"),
    MIDDLE("Sredina"),
    END("Kraj")
}

val w = Window.MIDDLE
println(w.label)         // Sredina
println(w.name)          // MIDDLE
println(Window.entries)  // [START, MIDDLE, END]

val hint = when (w) {
    Window.START -> "Prijavi se kad dođeš"
    Window.MIDDLE -> "Prijavi se nakon pauze"
    Window.END -> "Prijavi se prije odlaska"
}
```

KVIZ · ENUM KLASE

Ulaz dolazi iz tekstualne datoteke

```
println(Window.END.ordinal)         // 1
println(Window.valueOf("MIDDLE"))   // 2
println(Window.valueOf("middle"))   // 3
```

Šta se dešava u svakoj liniji?

Linija 3 je stvarni izgled ulaza. Kako biste je učinili sigurnom?

ODGOVOR

2 · MIDDLE · crash

```
// 3: IllegalArgumentException – imena razlikuju velika i mala slova

// Sigurno: vraća Window? umjesto pada
fun parseWindow(text: String): Window? =
    Window.entries.find {
        it.name.equals(text.trim(), ignoreCase = true)
    }

parseWindow(" middle ")  // MIDDLE
parseWindow("break")     // null
```

ordinal počinje od 0.

Pravilo: nikad ne vjeruj ulazu. Vrati **null** i neka developer odluči šta dalje.

4 · SEALED INTERFEJSI

Fiksan skup slučajeva, svaki sa svojim podacima

- **enum**: svaka vrijednost ima isti oblik
- **sealed**: Late ima minute, ostali nemaju
- **data object** = jedan slučaj, bez podataka
- **is** provjerava slučaj, zatim **smart cast**
- Bez **else**: kompajler provjerava da ste pokrili sve

Android: svako stanje ekrana je **Loading**, **Success(data)** ili **Error(message)**.

```
sealed interface CheckInStatus {
    data object OnTime : CheckInStatus
    data class Late(val minutes: Int) : CheckInStatus
    data object Absent : CheckInStatus
}

fun label(s: CheckInStatus): String = when (s) {
    CheckInStatus.OnTime -> "Na vrijeme"
    is CheckInStatus.Late -> "Kasni ${s.minutes} min"
    CheckInStatus.Absent -> "Odsutan"
}

// s.minutes radi: unutar te grane
// s je smart-cast u Late
```

KVIZ · SEALED INTERFEJSI

Stiže novi status. Šta ne radi?

```
sealed interface CheckInStatus {
    data object OnTime : CheckInStatus
    data class Late(val minutes: Int) : CheckInStatus
    data object Absent : CheckInStatus
    data class Excused(val reason: String) : CheckInStatus
}

// label() sa prethodnog slajda: nepromijenjen
```

A: Ništa, kompajlira se

B: Pad pri izvršavanju

C: Greška pri kompajliranju u label()

A da label() ima **else** granu?

ODGOVOR

C: kompajler vas zaustavlja, i to je dobro

```
// Error: 'when' expression must be exhaustive.
// Add the necessary 'is Excused' branch.

is CheckInStatus.Excused -> "Opravdano: ${s.reason}"

// Sa else -> "Odsutan" kompajlira se...
// ...i svaki opravdani student je prikazan kao Odsutan.
```

Kompajler navodi **svako mjesto** koje treba ažurirati.

else na sealed tipu skriva greške. AI ga često dodaje.

5 · INTERFEJSI

Ugovor: šta, ne kako

- **interface** navodi funkcije koje treba obezbijediti
- **: CheckInRule** = „implementiram ga“
- **override** je obavezan
- Kod koristi interfejs, ne klasu

Danas ćete vidjeti: **class MainActivity : ComponentActivity()**. Sa **()** nasljeđuje klasu; bez njih implementira interfejs.

```
interface CheckInRule {
    fun isValid(c: CheckIn): Boolean
}

class OnTimeRule(val limit: Int) : CheckInRule {
    override fun isValid(c: CheckIn) = c.minute <= limit
}

class RoomRule(val room: String) : CheckInRule {
    override fun isValid(c: CheckIn) = c.room == room
}

val rules: List<CheckInRule> = listOf(OnTimeRule(615), RoomRule("A1"))
val ok = rules.all { it.isValid(checkIn) }
```

KVIZ · INTERFEJSI

Ovo se ne kompajlira. Pronađite dvije greške

```
class LateRule : CheckInRule() {
    fun isValid(c: CheckIn) = c.minute > 615
}
```

Dvije greške, dvije linije.

ODGOVOR

Bez () iza interfejsa, a override je obavezan

```
class LateRule : CheckInRule {
    override fun isValid(c: CheckIn) = c.minute > 615
}
```

1. Interfejs nema konstruktor, pa nema **()**.

2. Bez **override** kompajler javlja da funkcija „hides member of supertype“.

DISKUSIJA

Oba su interfejsi. Zašto je samo jedan sealed?

```
sealed interface CheckInStatus {
    data object OnTime
    data class Late(val minutes: Int)
    data object Absent
}
```

```
interface CheckInRule {
    fun isValid(c: CheckIn): Boolean
}

class OnTimeRule ...
class RoomRule ...
```

I drugo: zašto je **Window** enum, a ne sealed interface?

SAŽETAK · IZBOR TIPA

Vrijednosti, slučajevi ili ponašanje?

`enum`

Fiksne vrijednosti, sve istog oblika

Window: START, MIDDLE, END. Svaka vrijednost ima ista polja.

`sealed interface`

Fiksni slučajevi, svaki svog oblika

CheckInStatus: OnTime, Late(minutes), Absent. Zatvoren: niko ne dodaje slučaj negdje drugo.

`interface`

Ugovor o ponašanju, otvoren

CheckInRule: bilo šta sa isValid(). Nova pravila bilo kad, kod koji ih koristi ostaje isti.

Pitajte se: je li lista zatvorena? Izgledaju li svi slučajevi isto? Opisujem li podatke ili ono što nešto radi?

6 · EXTENSION FUNKCIJE

Dodajte funkciju tipu koji nije vaš

- **fun Type.name()**
- Unutra je **this** sam objekat
- Poziva se kao obična metoda
- Podatak čuva broj, **label()** pravi tekst za ekran

Compose je izgrađen na njima: **Modifier.padding()** je extension funkcija na Modifier.

```
fun Student.initials(): String =
    name.split(" ")
        .map { it.first().uppercaseChar() }
        .joinToString("")

fun CheckInStatus.label(): String = when (this) {
    CheckInStatus.OnTime -> "Na vrijeme"
    is CheckInStatus.Late -> "Kasni $minutes min"
    CheckInStatus.Absent -> "Odsutan"
}

amar.initials()                // "AH"
CheckInStatus.Late(5).label()  // "Kasni 5 min"
```

KVIZ · EXTENSION FUNKCIJE

Ista funkcija, tri imena

```
Student("1", "lejla kovač").initials()   // 1
Student("2", "Emina").initials()         // 2
Student("3", "Amar  Hodžić").initials()  // 3 (dva razmaka)
```

Jedan od njih puca. Koji i zašto?

ODGOVOR

"LK" · "E" · pad

```
"Amar  Hodžić".split(" ")
// [Amar, , Hodžić] – prazan string u sredini

"".first()
// NoSuchElementException: Char sequence is empty

// Popravka: izbaci prazne dijelove
name.split(" ").filter { it.isNotBlank() }
```

Radi na primjerima koje ste probali, puca na stvarnim podacima.

Sljedeće: kako sigurno čitati neuredan tekst.

7 · PARSIRANJE TEKSTA

Od linije teksta do objekta, ili null

lines() dijeli redove · **split(";")** dijeli polja · **trim()** uklanja razmake · **toIntOrNull()** vraća Int? · **?: return null** rano odustaje

```
fun parseTime(t: String): Int? {
    val parts = t.split(":")
    if (parts.size != 2) return null
    val h = parts[0].toIntOrNull()
        ?: return null
    val m = parts[1].toIntOrNull()
        ?: return null
    return h * 60 + m
}
```

```
val log = """
    IB101; Amar Hodžić; 10:07
    IB102; Lejla Kovač; deset i pet
""".trimIndent()

fun parseLine(line: String): CheckIn? {
    val p = line.split(";").map { it.trim() }
    if (p.size != 3) return null
    val minute = parseTime(p[2]) ?: return null
    return CheckIn(Student(p[0], p[1]), minute)
}

val ok = log.lines().mapNotNull { parseLine(it) }
// 1 prijava: druga linija je preskočena
```

KVIZ · PARSIRANJE

Rezultat, null ili pad?

```
"42".toIntOrNull()          // 1
" 42".toIntOrNull()         // 2
"4.5".toIntOrNull()         // 3
"abc".toInt()               // 4
"a;b;;c".split(";").size    // 5
```

ODGOVOR

42 · null · null · pad · 4

```
" 42".toIntOrNull()        // null – razmaci se ne uklanjaju
" 42".trim().toIntOrNull() // 42

"abc".toInt()
// NumberFormatException

"a;b;;c".split(";")
// [a, b, , c] – prazna polja ostaju
```

Pravilo: **trim()** prvo, zatim **toIntOrNull()**. Nikad **toInt()** na ulazu koji ne kontrolišete.

Odgovori na pitanja o listi, bez petlji

```
data class CheckIn(val student: Student, val window: Window, val minute: Int, val status: CheckInStatus)
```

checkIns. …	Vraća	Na koje pitanje odgovara
`filter { it.status is CheckInStatus.Late }`	`List<CheckIn>`	Ko je kasnio?
`map { it.student.name }`	`List<String>`	Samo imena
`count { it.window == Window.MIDDLE }`	`Int`	Koliko ih je u sredini?
`sortedBy { it.minute }`	`List<CheckIn>`	Najraniji prvi
`maxByOrNull { it.minute }`	`CheckIn?`	Posljednji koji je stigao
`groupBy { it.window }`	`Map<Window, List<CheckIn>>`	Prijave po terminu
`map { it.minute }.average()`	`Double`	Prosječno vrijeme dolaska

Android: upravo ovakvu listu pripremate prije prikaza u **LazyColumn**.

KVIZ · KOLEKCIJE

Niko se nije prijavio u terminu END

```
val byWindow = checkIns.groupBy { it.window }

println(byWindow[Window.START]?.size)  // 12
println(byWindow[Window.END])          // ?
```

1. Kojeg je tipa byWindow?
2. Šta ispisuje zadnja linija?
3. Ispišite broj za svaki termin, uključujući 0.

ODGOVOR

Ključ koji ne postoji je null, ne prazna lista

```
// 1: Map<Window, List<CheckIn>>
// 2: null – groupBy pravi samo ključeve koji se pojave

// 3:
for (w in Window.entries) {
    val n = byWindow[w]?.size ?: 0
    println("${w.label}: $n")
}
// Početak: 12 · Sredina: 9 · Kraj: 0
```

map[key] uvijek vraća nullable tip.

Iterirajte kroz **Window.entries** da ništa ne bude preskočeno.

CODE REVIEW · U PAROVIMA, 5 MINUTA

AI je napisao naš model. Da li biste objavili ovaj kod?

```
class CheckIn(var studentId: String, var window: Int,
              var status: String)

fun lateCount(list: List<CheckIn>) =
    list.filter { it.status == "late" }.size

fun describe(c: CheckIn) = when (c.status) {
    "onTime" -> "Na vrijeme"
    "late" -> "Kasni"
    else -> "Odsutan"
}

val a = CheckIn("IB101", 2, "Late")
println(a == CheckIn("IB101", 2, "Late"))
```

Kompajlira se i radi.

Pronađite barem **4 problema**. Za svaki: šta ide naopako i koji današnji koncept to popravlja?

RJEŠENJE

Pet problema, današnji koncepti rješavaju sve

Problem	Popravka
Obična klasa: == ispisuje false	data class
Sve je var	val
"Late" ≠ "late": broj je 0	sealed CheckInStatus
else: greška u kucanju postaje Absent	when bez else
window = 7 je dozvoljeno	enum Window

```
data class CheckIn(
    val studentId: String,
    val window: Window,
    val status: CheckInStatus
)

fun lateCount(list: List<CheckIn>) =
    list.count {
        it.status is CheckInStatus.Late
    }
```

Stringovi za kategorije su najčešća AI prečica. Kompajler vam tu ne može pomoći.

PAUZA · 10 MINUTA

Android

DIO 2

Hello Android

ŠTA JE ANDROID APLIKACIJA

Četiri riječi koje ćete slušati svake sedmice

Activity

Ulazna tačka koju sistem pokreće. Uz Compose obično jedna Activity za cijelu aplikaciju.

Manifest

AndroidManifest.xml: lična karta aplikacije. Ime, ikona, aktivnosti, dozvole.

Gradle

Alat za build. Kompajlira Kotlin, preuzima biblioteke, pakuje aplikaciju.

APK i AAB

APK se instalira na telefon. AAB je ono što postavljate na Google Play.

VERZIJE KAO ODLUKA

Tri broja u svakom projektu

`minSdk`

Najstariji Android koji može instalirati vašu aplikaciju. Niže = više telefona, više posla oko kompatibilnosti.

`targetSdk`

Verzija prema kojoj ste testirali. Google Play svake godine postavlja minimum.

`compileSdk`

API-ji prema kojima kompajlirate. Obično najnoviji.

API nivo	24	26	29	33	35	36	37
Android	7.0	8.0	10	13	15	16	17

Wizard za New Project pokazuje koji udio uređaja pokriva svaki minSdk. Provjerite tamo, mijenja se svake godine.

KVIZ · VERZIJE

Naša aplikacija za prisustvo: minSdk 24 ili 34?

`minSdk = 24`

Android 7.0 i noviji

`minSdk = 34`

Android 14 i noviji

Mislite na svakog studenta u ovoj sali. Šta vas košta svaki izbor?

ODGOVOR

Aplikacija za prisustvo mora raditi na telefonu svakog studenta

24: skoro svi

Uključeni i stariji i jeftiniji telefoni. Cijena: noviji API-ji traže provjeru verzije.

34: samo najnoviji API-ji

Jednostavniji kod. Cijena: studenti sa starijim telefonima se uopšte ne mogu prijaviti.

Na ovom predmetu: **minSdk 24**, targetSdk i compileSdk = **36**.

OBILAZAK PROJEKTA · NEW PROJECT → EMPTY ACTIVITY

Gdje se šta nalazi

```
app/
  src/main/
    AndroidManifest.xml    // lična karta aplikacije
    java/ba/untz/attendance/
      MainActivity.kt      // ulazna tačka
      ui/theme/            // Color, Theme, Type
    res/                   // slike, stringovi, ikona
  build.gradle.kts        // SDK verzije, biblioteke
gradle/libs.versions.toml // sve verzije biblioteka
settings.gradle.kts
```

Kotlin kod je u folderu **java/**. Historijski naziv.

libs.versions.toml je mjesto gdje provjeravate šta bi AI-generisani Gradle kod promijenio.

HELLO WORLD · MAINACTIVITY.KT

Sve ovo već znate pročitati

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AttendanceTheme {
                Text("Zdravo RMAS!")
            }
        }
    }
}
```

: ComponentActivity() nasljeđuje klasu

override sistem poziva onCreate

Bundle? null pri prvom pokretanju

setContent { } trailing lambda

Text(...) naš prvi composable

Pokrenite na emulatoru, pa na svom telefonu.

ANDROID U 2026.

Dva pravila koja utiču na vaš projekat

Target API 36

Od 31. augusta 2026. nove aplikacije i ažuriranja na Google Playu moraju ciljati **Android 16 (API 36)** ili noviji.

Minimum raste svake godine. Stari tutorijali i AI često koriste stare vrijednosti.

Verifikacija developera

Aplikacije moraju biti od registrovanog developera. Od 30. 9. 2026. u nekoliko zemalja, **globalno 2027**.

Pokretanje iz Android Studija preko ADB-a je izuzeto. Za dijeljenje APK-a: besplatan studentski nalog, do 20 uređaja.

Izvori: developer.android.com · Target API level requirements; Android developer verification FAQ

DIO 3

Uvod u Jetpack Compose

PROMJENA PARADIGME

Ne mijenjajte ekran. Opišite ga

Imperativno: XML view-ovi (prije)

```
<!-- layout.xml -->
<TextView android:id="@+id/status" />

// MainActivity.kt
val status = findViewById<TextView>(R.id.status)
if (late) status.text = "Kasni"
else status.text = "Na vrijeme"
// zaboravite jedno ažuriranje: ekran je pogrešan
```

Deklarativno: Compose (sada)

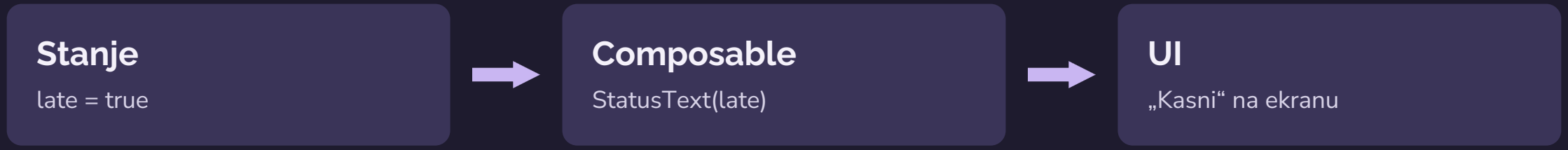
```
@Composable
fun StatusText(late: Boolean) {
    Text(if (late) "Kasni" else "Na vrijeme")
}

// jedan jezik, bez XML-a, bez findViewById
// late se promijeni: funkcija se ponovo izvršava
```

Vi opisujete **šta** UI prikazuje za date podatke. Compose sam određuje **kako** da ga ažurira.

OSNOVNA IDEJA

UI = f(stanje)



Rekompozicija: stanje se promijeni, Compose ponovo poziva pogođene composable funkcije.

Jednosmjerni tok podataka: stanje teče prema dolje, događaji (klikovi) prema gore.

COMPOSABLE FUNKCIJE

Funkcija koja emituje UI

- Označena sa **@Composable**
- Ne vraća ništa, nego **emituje** UI
- Imenuje se u **PascalCase**, kao **imenica**
- Poziva se samo iz drugih composable-a
- Podaci ulaze kao **parametri**

Kompozicija umjesto nasljeđivanja: ne nasljeđuje se Button. Male composable funkcije kombinujete u veće.

```
// obična funkcija vraća vrijednost
fun lateCount(list: List<CheckIn>): Int = ...

// composable funkcija opisuje UI
@Composable
fun Greeting(name: String) {
    Text("Zdravo $name!")
}

@Composable
fun StudentHeader(student: Student) {
    Column {
        Greeting(student.name)
        Text(student.id)
    }
}
```

KVIZ · IMENOVANJE

Koja imena poštuju pravila?

```
// sve označene sa @Composable
A  fun CheckInButton(text: String)
B  fun checkInButton(text: String)
C  fun RenderStudentCard(s: Student)
D  fun StudentAvatar(s: Student)
E  fun drawWindowLabel(w: Window)
```

Pravilo: PascalCase imenica koja opisuje šta nešto *jeste*.

ODGOVOR

A i D su ispravni

Ime	OK?	Zašto
CheckInButton	Da	PascalCase imenica
checkInButton	Ne	camelCase
RenderStudentCard	Ne	glagol: „render“ je ono što Compose radi
StudentAvatar	Da	PascalCase imenica
drawWindowLabel	Ne	camelCase i glagol

Lint u Android Studiju upozorava na ovo. AI kod to često ignoriše.

@PREVIEW

Pogledajte composable bez pokretanja aplikacije

- Preview funkcija **nema parametara**
- Poziva vaš composable sa **probnim podacima**
- Više previewa = više stanja odjednom
- Osvježava se u Design / Split prikazu

Preview nije aplikacija. Uvijek testirajte i na uređaju.

```
@Preview(showBackground = true, name = "Na vrijeme")
@Composable
fun StatusTextOnTimePreview() {
    StatusText(late = false)
}

@Preview(showBackground = true, name = "Kasni")
@Composable
fun StatusTextLatePreview() {
    StatusText(late = true)
}
```

OSNOVNI COMPOSABLE ELEMENTI

Gradivni blokovi koje ćete koristiti svaki dan

Text

```
Text("Amar Hodžić")
```

Image

```
Image(painterResource(R.drawable.logo), "Logo")
```

Button

```
Button(onClick = { }) { Text("Prijavi se") }
```

Icon

```
Icon(Icons.Default.Check, "Prisutan")
```

Spacer

```
Spacer(Modifier.width(12.dp))
```

TextField · sljedeće sedmice

Treba mu stanje da zapamti šta kucate

RASPOREDI

Column, Row, Box: sve ostalo se gradi od njih

Column

Elementi idu **jedan ispod drugog**

```
Column(
  verticalArrangement =
    Arrangement.spacedBy(8.dp)
) {
  Text("Ime")
  Text("10:07")
}
```

Row

Elementi idu **jedan pored drugog**

```
Row(
  verticalAlignment =
    Alignment.CenterVertically
) {
  Avatar()
  Text("Amar")
}
```

Box

Elementi idu **jedan preko drugog**

```
Box(
  contentAlignment =
    Alignment.Center
) {
  Image(...)
  Text("AH")
}
```

Arrangement = duž glavne ose. **Alignment** = poprijeko na nju.

RENDGEN COMPOSABLE-A · NA PAPIRU, 4 MINUTE

Skicirajte composable stablo ove poruke



Koji Row, Column i Box?

Koji Text i Image?

Nacrtajte kao stablo.